

CSE361 – Project Proposal

**Project 2: Multiplayer Game State
Synchronization**

Game Title: Sword Scuffle

Team : 36

Introduction:

Sword Scuffle is a 2D multiplayer fighting game that demonstrates real-time game-state synchronization over UDP.

The goal is to design a **lightweight, low-latency custom protocol** that keeps all players' positions, actions, and health values synchronized across clients while tolerating moderate packet loss and delay.

The project follows the “Multiplayer Game State Synchronization” scenario but applies it to a unique game idea

We will use **C# sockets inside Unity**

Motivation:

Real-time multiplayer games require fast updates and efficient state delivery. Using **UDP** allows very low latency, but it introduces packet loss and unordered delivery.

Sword Scuffle gives us a good testing ground to:

- Design a custom synchronization protocol (**SSTP**).
- Handle **loss, jitter, and delay** gracefully.
- Keep the gameplay smooth through **Delta encoding** and **selective reliability** for critical events.

The project connects networking theory with real-world multiplayer design.

Game Concept Overview:

Genre: 2D fighting / arena combat.

Players: 2 – 4 fighters connected to a central server.

Goal: Players move, jump, and attack each other using swords. The last player standing wins.

Gameplay loop:

- Players send **movement and attack inputs** to the server.
- The server updates positions, detects hits, and broadcasts snapshots to all clients.
- Clients smoothly render the received states.

Proposed Protocol Approach:

Transport: UDP (for all game updates).

Message types:

- **SNAPSHOT** – server-to-clients state update (positions, health, animation IDs).
- **EVENT** – critical events (e.g., hit landed, round start).
- **ACK** – acknowledgment for critical events only (selective reliability).
- **Connect** – client connecting/disconnecting to the game
- **Keyframe** – snapshots that happen every set amount of time that contains all player data so clients are up to date
- **ID_SET** – server assigns an ID to the client

Reliability mechanism:

- Delta encoding (send only changes since last acknowledged snapshot),
- Critical events (kills, round end) use **selective ACK** for guaranteed delivery.

Synchronization rate: 60 updates per second (~16.67 ms interval).

Maximum payload: ≤ 1200 bytes (header + payload).

Preliminary Header Format:

Field	Size (bytes)	Description
Protocol ID	4	"LABA" – Sword Scuffle Protocol
Version	1	Protocol version
Msg Type	1	0 = INPUT, 1 = SNAPSHOT, 2 = EVENT, 3 = ACK
Snapshot ID	4	Identifier for game state update
Seq Num	4	Sequence number per connection
Timestamp (ms)	8	Server or client send time
Payload Length	n	Size of payload data
Checksum	4	CRC32 for error detection

Total ≈ 26 bytes + payload (positions, HP, etc.).

Communication Flow (simplified):

Client → Server: **INPUT**(move_x, move_y, attack)

Server → Clients: **SNAPSHOT**(snapshot_id, all players' positions, health)

Client → Server: **EVENT**(hit_confirmed)

Server → Clients: **EVENT_BROADCAST**(hit_result)

Github repo: <https://github.com/AhmedRamySaid/Sword-Scuffle>